EXERCISE -1

Aim:

Write a C program that contains a string (char pointer) with a value 'Hello World'. The program should XOR each character in this string with 0 and displays the result.

Description:

- The program stores a plaintext message ("Hello World") using a character pointer.
- Each character of the plaintext is processed individually.
- The XOR operation is applied between each character and the value 0.
- XOR with 0 leaves the original data unchanged, demonstrating the identity property of XOR.
- This shows that encryption without a secret key does not provide confidentiality.
- The program highlights the importance of using a non-zero secret key in cryptographic encryption.
- XOR is a fundamental operation used in many cryptographic algorithms and stream ciphers.
- The program serves as a basic conceptual model for understanding XOR-based encryption and decryption.

Cryptography Note

XOR with 0 does not protect the data, so a secret key is needed to keep information secure in cryptographic algorithms.

Algorithm

1. Start the program.
2. Declare& initialize the string "Hello World".
3. Declare an integer variable to use as a counter.
4. Read the first character of the string.
5. Perform the XOR operation between the character and 0.
6. Display the resulting character.
7. Increment the counter to move to the next character.
8. Repeat steps 4 to 7 until the null character ('\0') is encountered.
9. Stop the program.

Program:

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
int main()
{
char *str="Hello world";
int length,i;
char result;
clrscr();
length = strlen(str);

for(i=0;i<length;i++)
{
result = str[i]^0;
printf("%c",result);
}
getch();
return 0;
}
```

Output:

Hello world

EXERCISE -2

Aim:

Write a C program that contains a string (char pointer) with a value \Hello World'. The program should AND and XOR each character in this string with 127 and display the result

Description:

- The program stores a string **"Hello World"** in a character array.
- It uses **AND operation with 127** to show that the string remains unchanged.
- It uses **XOR operation with 127** to encrypt the string.
- The encrypted string appears unreadable, providing basic security.
- The same XOR operation is applied again to decrypt the string.
- The original string is successfully recovered after decryption.

Cryptography Note

- XOR hides the message using a key.
- AND does not hide the message.

The same key gets the message back.

Algorithm:

1. Start the program.
2. Store the message **"Hello World"** in a string.
3. Apply **AND with 127** to each character and display it.
4. Apply **XOR with 127** to encrypt the string.
5. Apply **XOR with 127** again to decrypt the string.
6. Display the final output.
7. Stop the program

Source Code:

```
#include<stdio.h>

#include<conio.h>

#include<string.h>

int main()

{
```

```
char *str="Hello world";
int length,i;
char result;
clrscr();
length = strlen(str);

for(i=0;i<length;i++)
{
result = str[i]^127;
printf("%c",result);
}
for(i=0;i<length;i++)
{
result = str[i]&127;
printf("%c",result);
}
getch();
return 0;
}
```

Output:

Hello world

Hello world

Hello world

EXERCISE -3(a)

Aim:

Write a Java program to perform encryption and decryption using the following algorithm: a) CeaserCipher

Description:

- One of the oldest encryption methods
- Used by Julius Caesar
- It shifts letters by a fixed number (key)
- Same shift is used for all letters

Example:

- Key = 3
- A → D, B → E, C → F
- HELLO → KHOOR

Formulas:

- Encryption: $C = (P + K) \bmod 26$

Decryption: $P = (C - K) \bmod 26$

Algorithm:

Step 1: Start

Step 2: Read text and shift value

Step 3: For each character in text

Shift the letter by given value
Keep other characters same

Step 4: Store result as encrypted text

Step 5: For each character in encrypted text

Shift back by same value

Step 6: Store result as decrypted text

Step 7: Print encrypted and decrypted text

Step 8: Stop

Program:

```
import java.util.*;
class CaserCipher
{
	public static void main(String args[])
	{
	Scanner sc = new Scanner(System.in); // adding keyboard to program
	String text,ctext="",dtext="";
	int shift;
	System.out.println("Enter text:");
	text = sc.nextLine(); // Hello

	System.out.println("Enter shift value:");
	shift = sc.nextInt(); //3
	for(int i=0;i<text.length();i++) // 0 to 4
	{
		char ch;
		ch = text.charAt(i); // H

		if(ch>='A'&&ch<='Z') //
		{
			ctext = ctext+ (char)((ch-'A'+shift)%26+'A'); // ""+(72-
65+3)%26 +65 = 10+65 = 75
		}
		else if(ch>='a'&&ch<='z') //
```

```
        {
                ctext = ctext+ (char)((ch-'a'+shift)%26+'a');           }
        else
        {
        ctext = ctext+ch;
        }
}
        for(int i=0;i<ctext.length();i++) // 0 to 4
{
        char ch;
        ch = ctext.charAt(i); // H

        if(ch>='A'&&ch<='Z') //
        {
                dtext = dtext+ (char)((ch-'A'+26-shift)%26+'A');
                // ""+(72-65+3)%26 +65 = 10+65 = 75
        }
        else if(ch>='a'&&ch<='z') //
        {
                dtext = dtext+ (char)((ch-'a'+26-shift)%26+'a');
}
        else
        {
        dtext = dtext+ch;
        }

}
```

```
        System.out.println("Encrypted text :"+ctext);
        System.out.println("Decrypted text :"+dtext);
        }
}
```

Output:

Enter text:

Hello

Enter shift value:

3

Encrypted text :Khoor

Decrypted text :Hello

EXERCISE -3(b)

Aim:

Write a Java program to perform encryption and decryption using the following algorithm: b) SubstitutionCipher

Description:

Substitution Cipher is an encryption method

- Each letter in the plaintext is replaced with another letter
- A fixed key (mapping) is used In your program:

- Alphabet → ABCDEFGHIJKLMNOPQRSTUVWXYZ
- Key → QWERTYUIOPASDFGHJKLZXCVBNM

So,

- A → Q
- B → W
- C → E
- and so on

Encryption: replace letters using key

Decryption: reverse the process

Algorithm:

Step 1: Start

Step 2: Read input text

Step 3: Convert text to uppercase

Step 4: For each letter in text

- Replace it using given key
- Keep space as it is

Step 5: Store as ciphertext

Step 6: For each letter in ciphertext

Replace it back using alphabet

Step 7: Store as decrypted text

Step 8: Print ciphertext and decrypted text

Step 9: Stop

Program:

```
import java.util.*;
class SubstitutionCipher
{
        public static void main(String args[])
        {
                Scanner sc = new Scanner(System.in);
                String text,ctext="",dtext="";
                String alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
                String key ="QWERTYUIOPASDFGHJKLZXCVBNM";
                System.out.print("Enter plain text:");
                text = sc.nextLine().toUpperCase();//ABC
                for(int i=0;i<text.length();i++) // A-0 B-1 C -2
                {
                        char ch;
                        ch = text.charAt(i); //A
                        for(int j=0;j<26;j++)
                        {
                                if(ch==alphabet.charAt(j))// A ==A ok
                                {
                                        ctext = ctext+key.charAt(j);// ""+Q
                                        break;
                                }
                        }
```

```
                if(ch==' ')
            {
                ctext = ctext +" ";
            }
        }
        System.out.println("Cipher text :"+ctext);
        for(int i=0;i<ctext.length();i++) // A-0 B-1 C -2
        {
            char ch;
            ch = ctext.charAt(i); //A
            for(int j=0;j<26;j++)
            {
                if(ch==key.charAt(j))// A ==A ok
                {
                    dtext = dtext+alphabet.charAt(j);// ""+Q
                    break;
                }

            }
                    if(ch==' ')
            {
                dtext = dtext +" ";
            }
        }
        System.out.println("Decrypted text :"+dtext);
    }
}
```

Output:

Enter plain text:Hello

Cipher text :ITSSG

Decrypted text :HELLO

EXERCISE -3(c)

Aim:

Write a Java program to perform encryption and decryption using the following algorithm: C) HillCipher

Description:

- Hill Cipher is a **matrix-based encryption technique**
- It uses **mathematics (matrix multiplication)** to encrypt text
- Letters are taken in pairs (2 at a time)
- Each pair is converted into numbers (A=0, B=1, … Z=25)
- A **key matrix** is used to generate ciphertext

In your program:

- Key matrix used:

|1 2|
|3 5|

- Decryption uses the **inverse matrix**:

|21 2|
| 3 25|

- If text length is odd → add **X** at end

Algorithm:

Step 1: Start
Step 2: Read input text
Step 3: Convert text to uppercase
Step 4: If text length is odd, add 'X'

Step 5: Take two letters at a time
Step 6: Convert letters to numbers (A = 0 to Z = 25)
Step 7: Apply encryption formula
Step 8: Convert numbers back to letters
Step 9: Store as encrypted text

Step 10: Take encrypted text in pairs
Step 11: Convert letters to numbers
Step 12: Apply decryption formula

Step 13: Convert numbers back to letters
Step 14: Store as decrypted text

Step 15: Print encrypted and decrypted text
Step 16: Stop

Program:

```
import java.util.Scanner;

public class HillCipher {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter text: ");

        String text = sc.next().toUpperCase();

        if (text.length() % 2 != 0)

            text += "X";

        String enc = "";

        String dec = "";

        // ENCRYPTION

        for (int i = 0; i < text.length(); i += 2) {

            int a = text.charAt(i) - 'A';

            int b = text.charAt(i + 1) - 'A';

            enc = enc + (char) ((a + 2 * b) % 26 + 'A');

            enc = enc + (char) ((3 * a + 5 * b) % 26 + 'A');

        }


        // DECRYPTION

        for (int i = 0; i < enc.length(); i += 2) {

            int a = enc.charAt(i) - 'A';

            int b = enc.charAt(i + 1) - 'A';

            dec = dec + (char) ((21 * a + 2 * b) % 26 + 'A');
```

```
            dec = dec +  (char) ((3 * a + 25 * b) % 26 + 'A');
        }
        System.out.println("Encrypted: " + enc);
        System.out.println("Decrypted: " + dec);
        sc.close();
    }
}
```

Output:

Output:

Enter text: HELLO

Encrypted: HIOZHN

Decrypted: HELLOX

EXERCISE -4

Aim:

Write a Java program to implement the DES algorithm logic

Description:

DES (Data Encryption Standard) is a symmetric key encryption algorithm.

- It takes 64-bit plaintext
- Uses a 56-bit key (in real DES)
- Performs 16 rounds
- Uses a Feistel structure (split into Left & Right)

our program is a simplified DES, not full DES, but follows the same idea:

- Split data into two halves
- Apply XOR and swapping repeatedly
- Combine to get ciphertext

Algorithm:

Step 1: Initialize 64-bit plaintext

Step 2: Split into L (32 bits) and R (32 bits)

Step 3:Initialize key (32 bits)

Step 4:

Repeat 16 rounds:

1. temp = R
2. R = L XOR (R XOR key)
3. L = temp

Step 5: Combine: Cipher = R + L

Step 6: Print encrypted output

Program:

```
public class SimpleDES {

    // XOR function

    static int[] xor(int[] a, int[] b) {
```

```
    int[] r = new int[a.length];
    for (int i = 0; i < a.length; i++)
        r[i] = a[i] ^ b[i];
    return r;
}
public static void main(String[] args) {
    // 64-bit plaintext (example)
    int[] plaintext = new int[64];
    for (int i = 0; i < 64; i++)
        plaintext[i] = i % 2;
    // Split into Left & Right
    int[] L = new int[32];
    int[] R = new int[32];
    System.arraycopy(plaintext, 0, L, 0, 32);
    System.arraycopy(plaintext, 32, R, 0, 32);

    // Simple key (same for all rounds)
    int[] key = new int[32];
    for (int i = 0; i < 32; i++)
        key[i] = 1;

    // 16 rounds
    for (int i = 0; i < 16; i++) {
        int[] temp = R;
        R = xor(L, xor(R, key)); // Feistel function
        L = temp;
    }
```

```
        // Combine result
        int[] cipher = new int[64];
        System.arraycopy(R, 0, cipher, 0, 32);
        System.arraycopy(L, 0, cipher, 32, 32);

        // Output
        System.out.println("Encrypted Text:");
      for (int i = 0; i < cipher.length; i++) {
    System.out.print(cipher[i]);
          }
    }
}
```

Output:

Encrypted Text:

1111111111111111111111111111111101010101010101010101010101010101

EXERCISE -5

Aim:

Write a C/JAVA program to implement the Blow Fish algorithm logic

Description:

Blowfish is a symmetric key block cipher designed by Bruce Schneier.

- Block size: 64 bits
- Key size: 32 to 448 bits
- Fast and secure encryption algorithm
- Uses Feistel structure (16 rounds)

In our Java program:

- Blowfish is implemented using Java's built-in Cipher class
- Encryption is done and output is shown in Base64 format

Algorithm:

Step 1: Read plaintext and secret key

Step 2: Convert the key into byte format and generate Blowfish key

Step 3: Initialize Blowfish cipher using Cipher.getInstance("Blowfish")

Step 4: Set cipher to encryption mode using secret key

Step 5: Divide plaintext into 64-bit blocks

Step 6: Perform key expansion to generate P-array and S-boxes

Step 7: For each block, perform 16 rounds:

a) Left = Left XOR P[i]

b) Right = F(Left) XOR Right

c) Swap Left and Right

Step 8: After 16 rounds:

a) Swap Left and Right

b) Right = Right XOR P[17]

c) Left = Left XOR P[18]

Step 9: Combine Left and Right to form encrypted block

Step 10: Convert encrypted bytes into Base64 format

Step 11: Display encrypted ciphertext

Program:

```
import javax.crypto.Cipher;

import javax.crypto.spec.SecretKeySpec;

import java.util.Base64;

public class BlowfishEncryption {

    public static void main(String[] args) {

        try {

            String plaintext = "Hello World";

            String keyString = "MySecretKey";   // Your own key


            // Create key

            SecretKeySpec key = new SecretKeySpec(keyString.getBytes(),
"Blowfish");


            // Create cipher

            Cipher cipher = Cipher.getInstance("Blowfish");


            // Initialize for encryption

            cipher.init(Cipher.ENCRYPT_MODE, key);


            // Encrypt

            byte[] encryptedBytes = cipher.doFinal(plaintext.getBytes());


            // Convert to Base64 for readable output

            String encryptedText =
Base64.getEncoder().encodeToString(encryptedBytes);
```

```
            System.out.println("Original Text: " + plaintext);
            System.out.println("Encrypted Text: " + encryptedText);

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Output:

Original Text: Hello World

Encrypted Text: +pf7S2q4OHpBv7sqXdXimg==

EXERCISE -6

Aim:

Write a C/JAVA program to implement the Rijndael algorithm logic.

Description:

Rijndael is a symmetric key encryption algorithm developed by Joan Daemen and Vincent Rijmen.

- Standardized as AES (Advanced Encryption Standard)
- Block size: 128 bits
- Key size: 128 / 192 / 256 bits
- Number of rounds:
  - 10 (128-bit key)
  - 12 (192-bit key)
  - 14 (256-bit key)

Algorithm:

Step 1: Read plaintext (128-bit block) and secret key

Step 2: Perform Key Expansion to generate round keys

Step 3: Arrange plaintext into 4×4 state matrix

Step 4: Initial Round:

AddRoundKey (XOR with first round key)

Step 5: Perform 9 Main Rounds:

a) SubBytes (byte substitution using S-box)

b) ShiftRows (row shifting)

c) MixColumns (column mixing)

d) AddRoundKey (XOR with round key)

Step 6: Final Round (10th round):

a) SubBytes

b) ShiftRows

c) AddRoundKey

(No MixColumns)

Step 7: Convert state matrix into ciphertext

Step 8: Display encrypted output

Program:

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;

public class AESExample {

    public static void main(String[] args) {
        try {
            String plaintext = "HelloWorld12345"; // 16 bytes
            String keyString = "MySecretKey12345"; // 16 bytes key

            // Create key
            SecretKeySpec key = new SecretKeySpec(keyString.getBytes(), "AES");

            // Create cipher
            Cipher cipher = Cipher.getInstance("AES");

            // Initialize for encryption
            cipher.init(Cipher.ENCRYPT_MODE, key);

            // Encrypt
            byte[] encryptedBytes = cipher.doFinal(plaintext.getBytes());
```

```
            // Convert to Base64
            String encryptedText =
Base64.getEncoder().encodeToString(encryptedBytes);


            System.out.println("Original Text: " + plaintext);
            System.out.println("Encrypted Text: " + encryptedText);


        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Output:

Original Text: HelloWorld12345

Encrypted Text: Z1r3xUQeK8n9pVg0hQhZ7A==

EXERCISE -7

Aim:

Using Java Cryptography, encrypt the text “Helloworld” using BlowFish

Create your own key using Java key tool.

Description:

Blowfish is a symmetric key encryption algorithm designed by Bruce Schneier.

It uses a 64-bit block size and variable key length (32 to 448 bits).

It is based on a Feistel network and performs 16 rounds of encryption.

Algorithm:

Step 1: Read plaintext ("Hello World") and secret key ("MySecretKey")

Step 2: Convert the key string into byte format and generate Blowfish key

Step 3: Initialize cipher using Cipher.getInstance("Blowfish")

Step 4: Set cipher to encryption mode using the secret key

Step 5: Divide plaintext into 64-bit blocks

Step 6: Perform key expansion to generate P-array (18 subkeys) and 4 S-boxes

Step 7: For each block, perform 16 rounds:

a) Left = Left XOR P[i]

b) Right = F(Left) XOR Right

c) Swap Left and Right

Step 8: After 16 rounds:

a) Swap Left and Right

b) Right = Right XOR P[17]

c) Left = Left XOR P[18]

Step 9: Combine Left and Right to form encrypted block

Step 10: Convert encrypted bytes into Base64 format

Step 11: Display encrypted ciphertext

Program:

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;
public class BlowfishEncryption {

    public static void main(String[] args) {
        try {
            String plaintext = "Hello World";
            String keyString = "MySecretKey";   // Your own key

            // Create key
            SecretKeySpec key = new SecretKeySpec(keyString.getBytes(),
"Blowfish");

            // Create cipher
            Cipher cipher = Cipher.getInstance("Blowfish");

            // Initialize for encryption
            cipher.init(Cipher.ENCRYPT_MODE, key);

            // Encrypt
            byte[] encryptedBytes = cipher.doFinal(plaintext.getBytes());

            // Convert to Base64 for readable output
            String encryptedText =
Base64.getEncoder().encodeToString(encryptedBytes);
```

```
            System.out.println("Original Text: " + plaintext);
            System.out.println("Encrypted Text: " + encryptedText);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Output:

Original Text: Hello World

Encrypted Text: +pf7S2q4OHpBv7sqXdXimg==

EXERCISE -8

Aim:

Write a program to implement RSA Algorithm

Description:

The RSA algorithm is a public key cryptography technique used for secure data transmission. It uses two keys: a public key for encryption and a private key for decryption. The security of RSA is based on the difficulty of factoring large prime numbers.

In this program, the user enters two prime numbers to generate keys. Then a message is encrypted using the public key and decrypted using the private key.

Algorithm:

Step 1: Start the program

Step 2: Input two prime numbers p and q

Step 3: Compute
$n = p \times q$

Step 4: Compute Euler's Totient function
$\varphi(n) = (p - 1) \times (q - 1)$

Step 5: Choose public key exponent e such that
$1 < e < \varphi(n)$ and $\gcd(e, \varphi(n)) = 1$

Step 6: Compute private key d such that
$d = e^{-1} \bmod \varphi(n)$

Step 7: Display public key (e, n) and private key (d, n)

Step 8: Input message M (number form)

Step 9: Encrypt the message using
$C = M^e \bmod n$

Step 10: Decrypt the message using
$M = C^d \bmod n$

Step 11: Display encrypted and decrypted message

Step 12: Stop

Program:

```
import java.math.BigInteger;
import java.util.Scanner;

public class RSAExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Step 1: Choose two prime numbers
        System.out.print("Enter prime number p: ");
        BigInteger p = sc.nextBigInteger();

        System.out.print("Enter prime number q: ");
        BigInteger q = sc.nextBigInteger();

        // Step 2: Compute n = p * q
        BigInteger n = p.multiply(q);

        // Step 3: Compute phi(n) = (p-1)*(q-1)
        BigInteger phi =
(p.subtract(BigInteger.ONE)).multiply(q.subtract(BigInteger.ONE));

        // Step 4: Choose e (public key exponent)
        System.out.print("Enter public key e: ");
        BigInteger e = sc.nextBigInteger();

        // Step 5: Compute d (private key exponent)
        BigInteger d = e.modInverse(phi);
```

```
        System.out.println("\nPublic Key (e, n): (" + e + ", " + n + ")");
        System.out.println("Private Key (d, n): (" + d + ", " + n + ")");

        // Step 6: Encryption
        System.out.print("\nEnter message (number): ");
        BigInteger message = sc.nextBigInteger();

        BigInteger cipher = message.modPow(e, n);
        System.out.println("Encrypted message: " + cipher);

        // Step 7: Decryption
        BigInteger decrypted = cipher.modPow(d, n);
        System.out.println("Decrypted message: " + decrypted);
        sc.close();
    }
}
```

Output:

Enter prime number p: 3

Enter prime number q: 3

Enter public key e: 3

Public Key (e, n): (3, 9)

Private Key (d, n): (3, 9)

EXERCISE -9

Aim:

Implement the Diffiec- Hellman Key Exchange mechanism using HTML and Java Script, consider the end of the parties (Alice) and the JavaScript application as other party(bob)

Description:

Diffie-Hellman Key Exchange is a method used to securely share a secret key between two parties over an insecure network. It does not send the actual secret key; instead, both parties generate the same key using mathematical operations.

In this program, the user provides a prime number (p), base (g), and Alice's private key. Bob's private key is generated randomly. Both compute their public keys and then independently generate a shared secret key, which will be the same.

Algorithm:

Step 1: Start the program

Step 2: Input prime number p

Step 3: Input base g

Step 4: Input Alice's private key a

Step 5: Generate Bob's private key b randomly

Step 6: Compute Alice's public key $A = g^a \bmod p$

Step 7: Compute Bob's public key $B = g^b \bmod p$

Step 8: Alice computes shared key $K = B^a \bmod p$

Step 9: Bob computes shared key $K = A^b \bmod p$

Step 10: Compare both keys (they should be equal)

Step 11: Display public keys and shared secret key

Step 12: Stop

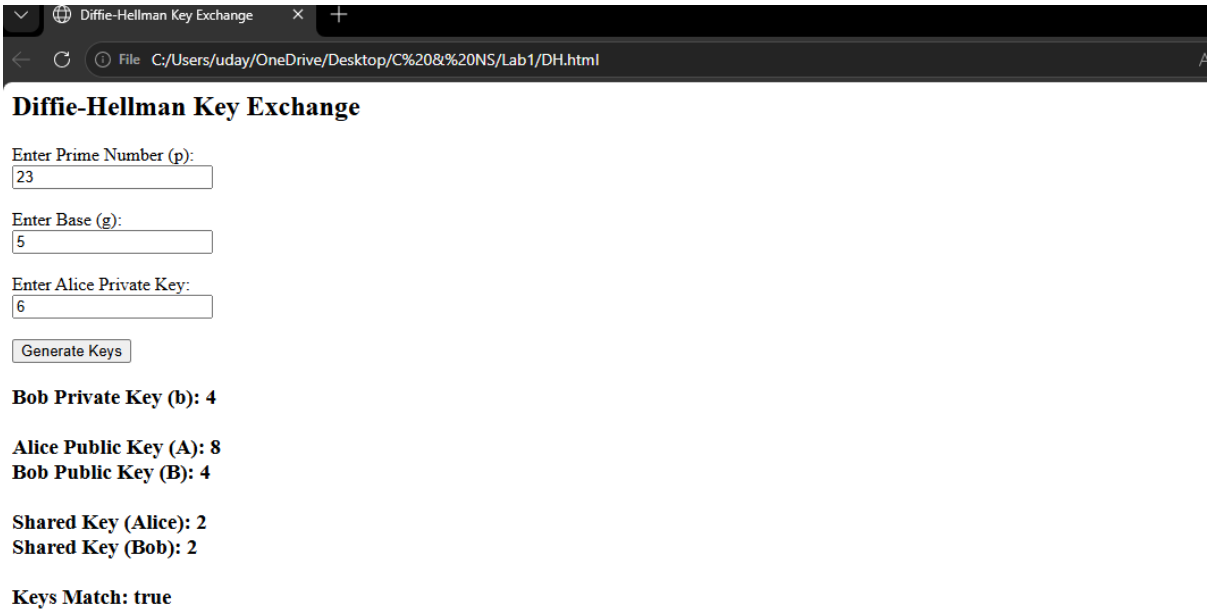
Program:

```
<html>
<head>
   <title>Diffie-Hellman Key Exchange</title>
</head>
<body>
<h2>Diffie-Hellman Key Exchange</h2>

<label>Enter Prime Number (p):</label><br>
<input type="number" id="p" value="23"><br><br>

<label>Enter Base (g):</label><br>
<input type="number" id="g" value="5"><br><br>

<label>Enter Alice Private Key:</label><br>
<input type="number" id="a" value="6"><br><br>

<button onclick="runDH()">Generate Keys</button>

<h3 id="output"></h3>

<script>
function power(base, exp, mod) {
   let result = 1;
   for (let i = 0; i < exp; i++) {
      result = (result * base) % mod;
   }
```

```
    return result;
}

function runDH() {
    let p = parseInt(document.getElementById("p").value);
    let g = parseInt(document.getElementById("g").value);
    let a = parseInt(document.getElementById("a").value);

    // Bob private key (random)
    let b = Math.floor(Math.random() * 10) + 1;

    // Public keys
    let A = power(g, a, p); // Alice public key
    let B = power(g, b, p); // Bob public key

    // Shared secret
    let keyAlice = power(B, a, p);
    let keyBob = power(A, b, p);

    document.getElementById("output").innerHTML =
        "Bob Private Key (b): " + b + "<br><br>" +
        "Alice Public Key (A): " + A + "<br>" +
        "Bob Public Key (B): " + B + "<br><br>" +
        "Shared Key (Alice): " + keyAlice + "<br>" +
        "Shared Key (Bob): " + keyBob + "<br><br>" +
        "<b>Keys Match: " + (keyAlice === keyBob) + "</b>";
}
```

```
</script>


</body>
</html>
```

Output:



EXERCISE -10

Aim:

Calculate the message digest of a text using SHA –1 algorithm in JAVA

Description:

SHA-1 (Secure Hash Algorithm 1) is a cryptographic hash function used to generate a fixed-size (160-bit) message digest from any input message. It is commonly used for data integrity and security purposes.

In this program, a message ("Hello World") is converted into a hash value using SHA-1. The message is first converted into bytes, processed by the MessageDigest class, and the resulting hash is displayed in hexadecimal format.

Algorithm:

Step 1: Start the program

Step 2: Define the input message (e.g., "Hello World")

Step 3: Create a MessageDigest object using SHA-1 algorithm

Step 4: Convert the input message into a byte array

Step 5: Generate the message digest using the digest() method

Step 6: Convert the resulting byte array into hexadecimal format

- For each byte
- Convert to hex value
- Append leading zero if needed
- Store in string

Step 7: Display the original message

Step 8: Display the SHA-1 message digest

Step 9: Stop

Program:

```
import java.security.MessageDigest;

class Sha1Ex

{
```

```
public static void main(String args[])
{
        try
        {
        String msg = "Hello World"; // input message
        MessageDigest md=  MessageDigest.getInstance("SHA-1"); //
create message digest object
        byte[] msgbytes = msg.getBytes(); // convert msg to byte array

        byte[] digestbytes = md.digest(msgbytes); // compute digest

        StringBuilder hexstring = new StringBuilder(); // convert byte array
to hex
                for(byte b: digestbytes )
                {
                        String hex = Integer.toHexString(0xff &b);

                        if(hex.length()==1)
                        {
                        hexstring.append('0');
                        }
                        hexstring.append(hex);
                }

        System.out.println("Original Message: "+msg);
        System.out.println("Message Digest: "+hexstring.toString());
        }
        catch(Exception e)
```

```
            {
            }
        }
}
```

Output:

Original Message: Hello World

Message Digest: 0a4d55a8d778e5022fab701977c5d840bbc486d0